Encoding Categorical Data

What's Covered

1. Why Encode Categorical Data?

Machine learning algorithms work with numbers, not text. Any column containing labels — like 'Mumbai', 'Low', 'Yes' — must be converted into a numeric representation before it can be fed into a model. This conversion process is called encoding.

- Without encoding, most sklearn models will raise an error when they encounter a text column.
- The CHOICE of encoding technique matters — using the wrong one can introduce a false sense of order or mislead the model.

Rule of thumb: the right encoding technique depends entirely on whether the categories have a natural order or not.

2. Types of Categorical Features — Nominal vs Ordinal

Type	Description & Example
Nominal	Categories with NO natural order. Example: City = {Mumbai, Delhi, Pune} — none is 'greater' than another.
Ordinal	Categories WITH a clear, meaningful order. Example: Education = {School < Graduate < Postgraduate}.

Ordinal Encoding and Label Encoding are best suited for ordinal or naturally rank-able data. Using them on purely nominal data can trick the model into believing one category is numerically 'greater' than another, which is meaningless and harmful.

3. Ordinal Encoding — Concept & Formula

Definition: Ordinal Encoding converts each category into an integer that reflects its natural rank or order. The category with the lowest rank gets 0, the next gets 1, and so on.

Because the assigned numbers preserve the real-world order, models that can exploit ordering — like linear models or tree-based splits — interpret the encoded values correctly.

3.1 Worked Example

Category	Assigned Integer
School	0
Graduate	1
Postgraduate	2

Notice: 2 > 1 > 0 correctly reflects Postgraduate > Graduate > School. The encoding is meaningful because the order was already meaningful in the original data.

4. OrdinalEncoder in Scikit-learn

Scikit-learn's OrdinalEncoder lets you explicitly define the order of categories using the categories parameter, ensuring the ranking is exactly what you intend — not alphabetical.

Python

```python
# Import the encoder
from sklearn.preprocessing import OrdinalEncoder

# Define the correct order explicitly (low to high)
edu_order = [['School', 'Graduate', 'Postgraduate']]

# Create encoder with the specified category order
encoder = OrdinalEncoder(categories=edu_order)

# Fit on training data — learns the mapping
encoder.fit(X_train[['Education']])

# Transform training and test data using the SAME mapping
X_train_enc = encoder.transform(X_train[['Education']])
X_test_enc = encoder.transform(X_test[['Education']])
```

4.1 Why Specify categories Explicitly?

- Without specifying categories, OrdinalEncoder sorts categories ALPHABETICALLY by default — which may not match the true real-world order.
- Example: alphabetically 'Graduate' < 'Postgraduate' < 'School' — completely wrong rank for education level.
- Always pass the correct order explicitly using the categories parameter for ordinal data.

CRITICAL: Always fit the encoder on training data only, then use transform() (not fit_transform()) on test data — exactly the same rule as with StandardScaler and MinMaxScaler, to avoid data leakage.

5. Label Encoding — Concept & Difference from Ordinal

Definition: Label Encoding assigns a unique integer (0, 1, 2, ...) to each distinct category, typically in alphabetical order. It is conceptually similar to Ordinal Encoding but is specifically designed for encoding the TARGET variable (y), not input features.

5.1 Key Difference: OrdinalEncoder vs LabelEncoder

OrdinalEncoder	LabelEncoder
Used on INPUT features (X)	Used on the TARGET variable (y)
Works on multiple columns at once (2D input)	Works on a single column only (1D input)
Lets you specify custom category order	Always assigns order alphabetically
Designed for feature preprocessing pipelines	Designed for encoding class labels

Common confusion: Many beginners use LabelEncoder on input features. It technically works, but it is intended for the target variable. For input features, prefer OrdinalEncoder (ordinal data) or One-Hot Encoding (nominal data, covered in Day 27).

6. LabelEncoder in Scikit-learn

Python

```python
# Import the encoder
from sklearn.preprocessing import LabelEncoder

# Create an instance
le = LabelEncoder()

# Fit on the target column and transform in one step
y_train_enc = le.fit_transform(y_train)

# Transform test labels using the SAME learned mapping
y_test_enc = le.transform(y_test)

# View the learned class order
print(le.classes_)
# e.g. array(['No', 'Yes'], dtype=object) -> No=0, Yes=1

# Reverse the encoding back to original labels
original_labels = le.inverse_transform(y_test_enc)
```

6.1 Practical Notes

- le.classes_ stores the sorted unique labels and their corresponding integer codes.
- inverse_transform() converts encoded integers back to original labels — useful for reporting predictions.

- If a new label appears in test data that wasn't seen during fit, LabelEncoder will raise an error.

7. Common Pitfalls

- ■ **Using Ordinal/Label Encoding on nominal data:** Introduces a fake order (e.g., City=2 > City=0) that misleads distance-based or linear models. Use One-Hot Encoding instead.
- ■ **Letting OrdinalEncoder sort alphabetically by default:** Always pass the categories parameter to enforce the true order.
- ■ **Using LabelEncoder on input features (X) with multiple columns:** LabelEncoder only handles one column at a time and has no concept of explicit ordering — use OrdinalEncoder for features.
- ■ **Fitting the encoder on the full dataset before splitting:** Causes data leakage — always fit on training data only.
- ■ **Unseen categories at prediction time:** Both encoders raise an error on unseen labels. Handle this with handle_unknown parameters or by ensuring consistent categories.

8. Summary

Concept	Key Takeaway
Why encode?	ML models need numbers — text categories must be converted first
Nominal data	No natural order (City, Color) — avoid Ordinal/Label Encoding
Ordinal data	Has natural order (Education, Rating) — Ordinal Encoding fits well
Ordinal Encoding	Maps ordered categories to integers reflecting their rank
OrdinalEncoder	sklearn class for INPUT features — specify categories explicitly
Label Encoding	Maps each unique category to an integer, typically for the TARGET
LabelEncoder	sklearn class for the TARGET (y) — single column, alphabetical order
Fit rule	Always fit on training data only, transform on test data
Avoid for nominal features	Use One-Hot Encoding instead (covered in Day 27)

Notes by: Amol Jagtap | amoljagtap3001@gmail.com | Day 26 — Encoding Categorical Data